

Cursor AI Training

AI-Assisted Software Engineering with Cursor AI Advanced Agentic Engineering Track

Total Duration: 32 Hours — 8 Half-Days (4 hours / half-day)

1. Agenda / Goal

- Provide developers with a comprehensive, engineering-lifecycle view of AI-assisted software engineering, using Cursor AI as the primary IDE and agentic development platform.
- Ground the course in a concise GenAI/LLM foundation before introducing tool-specific mechanics, so participants understand context, tool use, agents, and the limits of AI-assisted development.
- Enable teams to adopt AI-assisted practices across the full SDLC — requirements, design, specification, development, testing, review, deployment, and maintenance — not just code generation.
- Build proficiency with core and advanced Cursor capabilities: AI Chat, Tab/inline generation, codebase-aware editing, Ask, Agent, Plan, Debug, terminal/tool execution, rules, AGENTS.md, skills, subagents, and reusable workflows.
- Introduce Spec-Driven Development (SDD): treating a structured, versioned specification as the durable source of truth an agent implements against, with spec-to-code and spec-to-test traceability.
- Build reusable engineering assets using rules, skills, subagents, prompt templates, and repository standards, with clear separation of persistent instructions, specialized capabilities, delegated work, and task prompts.
- Introduce knowledge grounding and MCP for connecting agents to repositories, documentation, ticketing, APIs, and enterprise knowledge sources.
- Establish agentic workflow patterns with sequential/parallel delegation, quality gates, correction loops, hooks, automated validation, and human approval checkpoints.
- Instrument engineering workflows for observability, security, governance, token economics, and ROI to support enterprise rollout decisions.
- Apply the full stack to transform a ticket into a validated, independently reviewed, deployment-ready, traceable engineering report.
- Build hands-on proficiency through five progressively advanced use-case labs, culminating in an end-to-end capstone.

2. Prerequisites

- Working knowledge of at least one programming language (JavaScript/Python/Java/.Net preferred)
- Familiarity with Git and version control workflows
- Basic understanding of IDEs (VS Code experience helpful, not mandatory)
- Understanding of the team's existing codebase/repo structure and SRS/SDS documentation practices
- Basic familiarity with a ticketing tool (Jira or Azure DevOps) — used in capstone
- Basic familiarity with unit/API testing concepts (PyTest or equivalent helpful, not mandatory)
- Basic familiarity with software design artifacts (architecture diagrams, API contracts) — used in the Spec-Driven Development module
- Basic understanding of AI/LLM, agent, or multi-agent orchestration concepts

3. Software / System / License Requirements

- Cursor AI – latest stable version, Enterprise/Team seat with required Agent capabilities enabled (Windows/Mac/Linux)
- System requirements: Minimum 8GB RAM (16GB recommended), 5GB free disk space
- Git installed and configured; access to a sandbox/training repository (provided by facilitator)
- Node.js / Python (or relevant language runtime) installed, based on project stack
- PyTest (or equivalent test framework) installed for the test-generation and validation labs
- Sample SRS/SDS documents, spec/SDD templates, coding standards, and a historical defect log (sandbox dataset provided)

3. Software / System / License Requirements (continued)

- MCP server access configured for repository, documentation, and knowledge-source connections (sandbox instance provided — pre-stage before Half-Day 4)
- Read-access API token / sandbox project for Jira or Azure DevOps (provided by facilitator) for the capstone
- CI/CD sandbox (e.g., GitHub Actions) for the deployment-readiness portion of Module 19
- Stable internet connection (required for AI model and agent/tool calls)
- Company SSO/IT approval for Cursor AI and connected MCP/agent tooling (if applicable)

4. Module Overview — 8 Half-Day Schedule

Day 1 — Foundations — GenAI/LLM Refresher & Cursor AI Introduction

#	Module / Segment	Duration
0	Introduction and Pre-Assessment	60 mins
1	GenAI/LLM Foundations Refresher	45 mins
—	Break	15 mins
2	Introduction to Cursor AI	30 mins
3	Cursor Interface & Setup	30 mins
4	AI Chat, Context & Ask Mode	60 mins
	Day 1 total	240 mins

Day 2 — Cursor Power User & AI-Assisted Coding

#	Module / Segment	Duration
5	Inline Code Generation, Tab & Context	45 mins
6	Codebase-Aware Editing & Agent Mode	60 mins
—	Break	15 mins
7	Plan, Debug, Refactoring & Testing	60 mins
8	Rules, AGENTS.md, Skills & Team Standards	60 mins
	Day 2 total	240 mins

Day 3 — Design, SDD & Agent Architecture

#	Module / Segment	Duration
9	AI-Assisted Design & Spec-Driven Development (SDD)	90 mins
—	Break	15 mins
10	Agent, Skill & Subagent Architecture Fundamentals	45 mins
11	Use Case Lab 1: Reusable Agents, Prompts & Skills Framework	90 mins
	Day 3 total	240 mins

4. Module Overview — 8 Half-Day Schedule (continued)

Day 4 — Context Engineering, Knowledge Grounding, Governance & Observability

#	Module / Segment	Duration
12	Context Engineering, Knowledge Grounding & MCP	90 mins
—	Break	15 mins
13	Use Case Lab 2: Knowledge-Grounded Engineering Agent	90 mins
14	Governance, Security & Observability	45 mins
	Day 4 total	240 mins

Day 5 — Multi-Agent Orchestration Build

#	Module / Segment	Duration
15	Subagents & Orchestration Patterns Overview	45 mins
16	Use Case Lab 3: Multi-Agent Requirement-to-Test Automation	60 mins
—	Break	15 mins
16	Use Case Lab 3: Multi-Agent Requirement-to-Test Automation (cont..)	90 mins
17	Quality Gates, Hooks & Self-Correction Fundamentals	30 mins
	Day 5 total	240 mins

Day 6 — Self-Correction & Deployment Readiness

#	Module / Segment	Duration
18	Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates	120 mins
—	Break	15 mins
19	Git, CI/CD, Cloud Agents & Ticketing Integration for Agentic Workflows	90 mins
—	Capstone project intro	15 mins
	Day 6 total	240 mins

4. Module Overview — 8 Half-Day Schedule (continued)

Day 7 — Capstone Build

#	Module / Segment	Duration
—	Capstone kickoff & scoping	15 mins
20	Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot	210 mins
—	Break	15 mins
	Day 7 total	240 mins

Day 8 — Review, Demos & Program Wrap-Up

#	Module / Segment	Duration
—	Peer / AI-assisted review of capstone	45 mins
—	Demos & presentations	75 mins
21	Best Practices, Enterprise Rollout & ROI	60 mins
—	Wrap-up, feedback & Q&A;	30 mins
—	Break / Buffer / final polish	30 mins
	Day 8 total	240 mins

Total Duration: 32 hours (1,920 minutes) across 8 half-days.

5. Modules — Detailed Topics & Subtopics

Day 1 — Foundations: GenAI/LLM Refresher & Cursor AI Introduction

Module 1 — GenAI/LLM Foundations Refresher

Topics:

- What LLMs are and how they generate text (high-level, non-mathematical)
- Tokens, context windows, and why they constrain agent and tool design
- Prompting fundamentals: instruction clarity, few-shot framing, system vs. user prompts
- Capabilities and known limitations of generative AI (hallucination, knowledge cutoffs, non-determinism)
- How generative AI differs from traditional deterministic software
- The AI-assisted SDLC landscape — from prompt-driven coding to context engineering, agents, and multi-agent workflows

Subtopics:

- Brief landscape overview: major model families, IDE-integrated vs. chat-based tools
- Core terminology (LLM, context window, context engineering, agent, subagent, skill, orchestration, MCP, spec-driven development)
- Why engineering workflows require verification, traceability, and human judgment

Hands-on: Guided comparison of a raw LLM response vs. a codebase-grounded agent response — establishes why context and grounding matter.

Module 2 — Introduction to Cursor AI

Topics:

- What is Cursor AI and how it differs from traditional IDEs
- Cursor vs. GitHub Copilot vs. other AI coding tools
- Cursor's role across the AI-assisted SDLC — from coding assistance to agentic engineering workflows
- Local development vs. delegated/background agent execution — conceptual overview

Subtopics:

- Positioning Cursor within the AI-Assisted Software Engineering theme established in Module 1

Hands-on: Quick tour and account/license verification.

Module 3 — Cursor Interface & Setup

Topics:

- Installing/configuring Cursor AI; connecting the enterprise license/account
- Interface walkthrough (Editor, Sidebar, Command Palette, Chat/Agent Panel)
- Importing existing VS Code settings/extensions; workspace and project setup
- Understanding model selection and basic agent configuration

Subtopics:

- Connecting MCP servers (repository, docs, ticketing) at the workspace level
- Understanding workspace trust, permissions, and safe tool execution

Hands-on: Hands-on setup and workspace validation exercise.

Module 4 — AI Chat, Context & Ask Mode

Topics:

- Using AI Chat for code queries; file/folder/codebase context
- Using @-mentions to reference files, docs, symbols, and relevant project context
- Difference between Chat/Ask and Agent modes
- Context selection and scoping for reliable answers
- Managing conversation history and context limits

Subtopics:

- Effective prompt writing and context engineering for analysis tasks

Hands-on: Hands-on exercise and quiz to follow this module.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 2 — Cursor Power User & AI-Assisted Coding

Module 5 — Inline Code Generation, Tab & Context

Topics:

- AI-powered autocomplete (Tab-to-complete); inline generation via natural-language prompts
- Generating functions, classes, boilerplate, and targeted code changes
- Accepting, rejecting, editing, and validating suggestions
- Using surrounding code and open-file context effectively

Subtopics:

- Customizing autocomplete settings and choosing the right interaction for small vs. larger tasks

Hands-on: Hands-on exercise to follow this module.

Module 6 — Codebase-Aware Editing & Agent Mode

Topics:

- Using Agent mode for multi-file changes and feature implementation
- Understanding codebase indexing/search and repository context
- Making cross-file edits and reviewing/approving diffs
- Using terminal/tool execution safely during agentic changes
- Scoping AI edits safely in large projects

Subtopics:

- Context window limitations; task decomposition; keeping changes reviewable and reversible

Hands-on: Hands-on exercise implementing a small feature across multiple files.

Module 7 — Plan, Debug, Refactoring & Testing

Topics:

- Plan mode for complex tasks: explore → plan → review → implement
- Debug mode for diagnosing failures and validating hypotheses
- Refactoring code using natural-language instructions; handling legacy code
- Using AI to identify bugs, explain stack traces, and suggest/apply fixes
- Writing, executing, and debugging unit/API tests with AI assistance

Subtopics:

- Validating AI refactors against existing tests; using terminal output and test failures as context

Hands-on: Hands-on exercise and quiz to follow this module.

Module 8 — Rules, AGENTS.md, Skills & Team Standards

Topics:

- Project Rules in .cursor/rules/ and project-level AI instructions
- AGENTS.md and persistent repository guidance
- User/Team/Project instruction scope and precedence
- Skills for reusable specialized capabilities
- Creating reusable prompt templates and standard workflows
- Standardizing AI-assisted development practices across the team

Subtopics:

- Version-controlling rules and skills; keeping instructions concise, testable, and maintainable

Hands-on: Hands-on exercise. Direct foundation for Module 9 and Module 11.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 3 — Design, Spec-Driven Development & Agent Architecture

Module 9 — AI-Assisted Design & Spec-Driven Development (SDD)

Topics:

- Writing a spec as the durable source of truth for agent-generated code
- Spec-to-code and spec-to-test traceability; spec versioning and change management
- Contrasting Spec-Driven Development with prompt-driven and vibe-coding approaches
- Using Cursor to explore requirements and create implementation plans
- Architecture diagram generation from requirements; API/interface contract drafting
- Architecture Decision Records (ADRs); translating SRS → SDS with AI assistance

Subtopics:

- What belongs in a spec vs. a prompt; acceptance criteria as testable spec clauses; human approval before implementation

Hands-on: Draft a spec for a sample feature, generate an implementation plan, review it, and generate a first implementation pass against the approved spec.

Module 10 — Agent, Skill & Subagent Architecture Fundamentals

Topics:

- Agent vs. rule vs. prompt vs. skill vs. subagent: definitions and when to use each
- Anatomy of a reusable agent (role, inputs, tools, guardrails, outputs)
- Subagent delegation: focused work, isolated context, and result handoff
- Parameterized prompt-template design
- Repository-level rules vs. team-level skills libraries
- Designing reusable workflows around Cursor's native agent capabilities

Subtopics:

- Versioning and reviewing agent/skill definitions like code; avoiding unnecessary agent decomposition

Hands-on: Hands-on walkthrough of a sample agent/subagent definition.

Module 11 — Use Case Lab 1: Reusable Agents, Prompts & Skills Framework

Topics:

Lab objective: Create reusable engineering agents, parameterized prompt templates, repository rules, and skills for requirement analysis, test generation, validation, and documentation.

Activities:

- Define agent roles for requirement analysis, test generation, validation, and documentation
- Build parameterized prompt templates with explicit input/output contracts
- Package rules and skills into a shared, version-controlled library
- Create a focused subagent for an independent engineering task
- Test each asset independently against sample inputs

Deliverable: A reusable agent/skill library committed to the shared training repository.

Hands-on lab and peer review.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 4 — Context Engineering, Knowledge Grounding, Governance & Observability

Module 12 — Context Engineering, Knowledge Grounding & MCP

Topics:

- Context engineering: selecting, structuring, and scoping context for engineering tasks
- Grounding sources: repository code, SRS/SDS documents, standards, historical defects, and enterprise knowledge
- MCP overview: resources, tools, and connections to external systems
- Retrieval strategies: semantic search, structured lookups, and graph traversal where applicable
- Hallucination risk in engineering contexts and why it matters

Subtopics:

- Designing 'no unsupported facts' guardrails; citing sources in agent output
- Separating context/knowledge retrieval from tool/action execution
- Choosing direct repository context, rules/skills, or MCP based on the task

Hands-on: Hands-on walkthrough of a context-grounding and MCP configuration.

Module 13 — Use Case Lab 2: Knowledge-Grounded Engineering Agent

Topics:

Lab objective: Build an agent that uses repository code, SRS/SDS documents, standards, historical defects, and enterprise knowledge sources without inventing unsupported engineering facts.

Activities:

- Connect the agent to repository, document, and knowledge sources via MCP
- Configure retrieval scoping and source citation in agent responses
- Use rules/skills to constrain the agent's behavior
- Stress-test with out-of-scope questions to verify refusal/grounding behavior

Deliverable: A grounded engineering agent that cites its sources and declines unsupported claims.

Hands-on lab and peer review.

Module 14 — Governance, Security & Observability

Topics:

- Data privacy and IP handling for connected documents and knowledge sources
- Enterprise admin controls, audit logs, and access scoping for agent tool use
- Safe execution: approvals, permissions, sandboxing, and controlled tool access
- Compliance considerations (SOC2, GDPR) as applied to grounded agents
- Tracing and logging across multi-agent pipelines
- What to log at each agent handoff for debuggability and traceability
- Cost visibility per agent call/task

Subtopics:

- Approved vs. restricted knowledge sources; restricted file/folder access policies; secrets handling; observability tooling options

Hands-on: Hands-on exercise and quiz to follow this module.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 5 — Multi-Agent Orchestration Build

Module 15 — Subagents & Orchestration Patterns Overview

Topics:

- Sequential vs. parallel multi-agent pipelines; state and handoff between agents
- Designing agent roles for a pipeline (Validator, Builder, Generator, Reviewer)
- Subagent delegation and independent-context patterns
- Failure handling, retries, timeouts, and bounded execution
- Choosing orchestration granularity and avoiding unnecessary agent complexity

Subtopics:

- Observability/logging across agent handoffs; distinguishing Cursor-native capabilities from external workflow orchestration

Hands-on: Hands-on walkthrough of a sample pipeline diagram.

Module 16 — Use Case Lab 3: Multi-Agent Requirement-to-Test Automation

Topics:

Lab objective: Orchestrate Requirement Validator, Sequence Builder, Test Generator, API Validator, and Reviewer agents to convert an engineering requirement into validated executable tests.

Activities:

- Configure the Requirement Validator agent to check requirement completeness and testability
- Chain the Sequence Builder agent to derive a test sequence from the validated requirement
- Chain the Test Generator agent to produce executable tests (e.g., PyTest) from the sequence
- Chain the API Validator agent to check generated tests against the target API/interface
- Chain the Reviewer agent to independently review and sign off before tests are committed

Deliverable: An executable, validated test suite traced back to its source requirement.

Hands-on lab and peer review.

Module 17 — Quality Gates, Hooks & Self-Correction Fundamentals

Topics:

- Designing automated PASS/FAIL criteria between agent stages
- Correction-loop patterns: who retries, what feedback is reused, and what reruns downstream
- Hooks for validation, policy enforcement, logging, and lifecycle controls
- Human-in-the-loop approval checkpoints: where and why to place them
- Bounded retries and avoiding infinite correction loops

Subtopics:

- Logging gate decisions for traceability; fail-safe behavior when tools, agents, or validators fail

Hands-on: Hands-on walkthrough of a gate and hook configuration.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 6 — Self-Correction & Deployment Readiness

Module 18 — Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates

Topics:

Lab objective: Implement generator–validator–reviewer orchestration with automated PASS/FAIL gates, bounded correction loops, downstream reruns, hooks, and human approval checkpoints.

Activities:

- Add automated gates between the generator, validator, and reviewer agents from Lab 3
- Implement a correction loop that routes FAIL results back to the generator with structured feedback
- Configure downstream reruns so only affected stages re-execute
- Add validation hooks for selected engineering checks
- Add a human approval checkpoint before final sign-off

Deliverable: A bounded self-correcting pipeline with gate logs, validation evidence, and an approval trail.

Hands-on lab and peer review.

Module 19 — Git, CI/CD, Cloud Agents & Ticketing Integration for Agentic Workflows (EXPANDED)

Topics:

- Connecting Cursor AI/agents to Jira or Azure DevOps via MCP
- Parsing ticket fields into structured agent inputs
- Maintaining traceability from ticket to plan/spec to commit to test to report
- Git workflow integration (branches, commits, PRs) for agent-generated artifacts
- GitHub Actions pipeline basics; AI-assisted PR preparation and review
- Automated deployment-readiness gates
- Cloud/Background Agents: use cases, remote execution, test execution, and PR-oriented workflows
- Local vs. delegated agent execution and associated security considerations

Subtopics:

- Environment setup, secrets, network access, permissions, and human review for delegated/background work

Hands-on: Hands-on exercise to follow this module.

Capstone project intro: Explain the end-to-end ticket-to-report workflow, team roles, acceptance criteria, repository, and evaluation rubric.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 7 — Capstone Build

Module 20 — Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot (EXPANDED)

Topics:

Lab objective: Transform a Jira/Azure DevOps ticket into a requirement bundle, approved plan/spec, test sequence, executable tests, validation evidence, independent review, deployment-readiness check, and a traceable final engineering report.

Activities:

- Pull a sandbox ticket and extract a structured requirement bundle
- Explore the repository and draft an implementation plan; obtain human approval
- Draft a short spec note establishing acceptance criteria (per Module 9's SDD approach)
- Generate the engineering test sequence from the requirement bundle
- Generate PyTest cases and execute them against the sandbox target
- Produce a validation report and route it to an independent reviewer agent/subagent
- Run security/quality and deployment-readiness gates from Modules 17–19
- Optionally demonstrate delegated/background execution for an isolated task
- Compile a checklist and a traceable final engineering report linking ticket → plan → spec → sequence → tests → results → review → deployment readiness

Deliverable: A complete, traceable engineering report package, ready for stakeholder sign-off, including an observability/cost summary.

Hands-on lab and peer review.

Xebia — Cursor AI Training | AI-Assisted Software Engineering

Day 8 — Review, Demos & Program Wrap-Up

Module 21 — Best Practices, Enterprise Rollout & ROI

Topics:

- Do's and don'ts of AI-assisted and multi-agent software development; avoiding over-reliance
- Verifying AI/agent output for correctness, security, maintainability, and traceability
- Choosing the right Cursor interaction: Tab/inline, Ask, Plan, Agent, Debug, Skill, Subagent, or delegated/background agent
- Rolling out rules, skills, agent patterns, and quality gates across a team
- Token economics: cost-per-call and cost-per-task modeling; context-window cost implications of large codebase grounding
- Building an ROI case: time saved vs. token spend vs. defect reduction
- Using historical defect logs and production feedback to maintain and update agent grounding sources over time
- Establishing ownership, governance, versioning, review, and maintenance for shared AI engineering assets

Subtopics:

- Common pitfalls in orchestration and how to avoid them
- When not to use an agent or autonomous workflow
- Maintenance, monitoring, and continuous improvement as ongoing lifecycle phases, not a one-time handoff

Hands-on: Hands-on discussion.

Peer / AI-assisted review, demos & wrap-up: Peer/AI-assisted review of each capstone deliverable against the rubric; demos and presentations of each team's end-to-end engineering report; wrap-up, feedback & Q&A.;

Programme outcome: Participants leave with a practical Cursor-based AI-assisted engineering workflow spanning requirements, planning, specification, implementation, testing, review, governance, CI/CD, and deployment readiness.

6. AI-Assisted SDLC Mapping

SDLC Phase	Cursor / Agent Capability	Module
Requirements	Ask / Agent / MCP / Ticket integration	4, 9, 12, 19
Analysis	Context engineering / repository exploration / grounding	4, 6, 9, 12
Architecture	Plan / diagrams / ADRs / spec generation	9
Specification	Spec-Driven Development / acceptance criteria	9
Implementation	Tab / Inline / Agent / multi-file editing	5, 6
Refactoring	Agent / Debug / tests	7
Testing	Test generation / execution / validation agents	7, 16
Code Review	Reviewer agent/subagent / diffs / quality gates	17, 18
Security	Rules / hooks / permissions / security review	14, 17, 18
CI/CD	Git / PR / GitHub Actions / deployment gates	19
Deployment Readiness	Automated gates / human approval	18, 19, 20
Maintenance	Defect feedback / grounding updates / agent improvement	20, 21

Recommended engineering mental model

- **Prompt Engineering** → clear task instructions
- **Context Engineering** → provide the right repository, specification, standards, and knowledge
- **Agent Engineering** → define roles, tools, guardrails, skills, and subagents
- **Workflow Engineering** → orchestrate stages, gates, correction loops, hooks, and approvals
- **AI-Assisted SDLC** → apply the above systematically from requirement to deployment readiness

Design principle: Cursor-native features are taught first; broader agentic engineering patterns are demonstrated using Cursor together with MCP, Git/CI, scripts, and workflow mechanisms where appropriate. This keeps the curriculum practical without implying that every orchestration capability is a single built-in Cursor feature.